



MSBA503 YELP BIG DATA PIPELINE

Databricks Pipeline Parameter Guide

A visual operating manual for moving the Yelp Academic Dataset from Google Drive through Silver intelligence, flexible Gold cohorts, and dashboard-ready tables.

GOOGLE DRIVE	BRONZE	SILVER	GOLD	DASHBOARD
--------------	--------	--------	------	-----------

ONE DATABRICKS JOB. THREE DEPENDENT TASKS. TWO INDEPENDENT SAMPLE LIMITS.

The pipeline at a glance

The 2026 workflow is a single ordered Databricks Job. Each task owns one layer and passes a validated Delta table to the next task.

1 BRONZE TO SILVER	2 SILVER TO GOLD	3 DASHBOARD LAYER
Select + enrich Google Drive -> scoped Bronze tables Deterministic review selection before NLP Selected NLP families -> Delta Silver Task: bronze_to_silver_yelp	Focus + shape Silver -> business/user/date/industry filters Row sample or analytical aggregate Gold request clamps to eligible Silver rows Task: silver_to_gold_yelp	Summarize + serve Selected row-level Gold -> serving tables Review, business, and monthly views No new sampling or cohort selection Task: data_science_dashboard_yelp

The dependency chain is the product

Silver always contains the selected reviews with full feature engineering. Gold decides what analytical slice or grain is published. The dashboard never bypasses Gold, so its content always reflects the selected Gold configuration.

Set up the Databricks Job

Keep the 5.23 GB source corpus outside Git and outside the constrained workspace storage path. Google Drive supplies raw input; Unity Catalog governs access and Delta outputs.

1

Place all five raw files in one Google Drive folder

Use the original Yelp JSON-lines filenames for review, business, user, check-in, and tip.

2

Create a Unity Catalog Google Drive connection

Grant the Databricks identity access to the folder and record the connection name.

3

Place lexicons in a Unity Catalog Volume

The small lexicon assets remain separate from both Git and the large raw corpus.

4

Add this repository as a Workspace Git folder

Run the scripts directly from the connected repository revision.

5

Create three dependent Python-script tasks

Use Bronze-to-Silver, then Silver-to-Gold, then Data-science Dashboard.

6

Paste one JSON array into each Parameters box

Job parameters may be referenced with `{{job.parameters.name}}`.

Source separation

Raw Yelp data stays in Google Drive. Lexicons stay in a Unity Catalog Volume. Code stays in Git. Delta tables and audit records stay in Unity Catalog.

```
[ "--flag", "value", "--standalone-flag" ]
```

Job parameters + JSON grammar

Job parameters are reusable defaults. Each Python-script task still receives a JSON array of strings in its own Parameters textbox.

JOB-LEVEL DEFAULTS

Name	Default	Used by
google_drive_folder_url	REPLACE_WITH_GOOGLE_DRIVE_FOLDER_URL	Bronze to Silver
google_drive_connection	yelp_google_drive	Bronze to Silver
silver_sample_size	25000	Bronze to Silver
silver_sample_seed	222	Bronze to Silver
gold_sample_size	5000	Silver to Gold
gold_sample_seed	222	Silver to Gold

TASK PARAMETERS JSON

```
[
  "--google-drive-folder-url", "{{job.parameters.google_drive_folder_url}}",
  "--google-drive-connection", "{{job.parameters.google_drive_connection}}",
  "--catalog", "workspace",
  "--schema", "default",
  "--silver-sample-size", "{{job.parameters.silver_sample_size}}",
  "--nlp-component", "linguistic",
  "--nlp-component", "vader"
]
```

JSON rules

Every flag and value is a separate quoted item. Quote numbers. Repeat the entire flag/value pair for repeatable options. Use `{{job.parameters.name}}` only when that Job parameter exists. Notebook tasks use a different key/value interface.

Two independent sampling controls

One limit controls what reaches Silver and incurs NLP work. The other controls the row-level Gold dataset presented to analysis and the dashboard.

BEFORE SILVER - sample the workload

--silver-sample-size N --silver-sample-seed S

Selects N review IDs deterministically before joins and feature engineering. Related business, user, check-in, and tip rows are scoped to those reviews. Only selected reviews become Silver.

AFTER GOLD FILTERS - shape the output

--gold-sample-size N --gold-sample-seed S

Applies after business, user, industry, date, geography, star, sentiment, and emotion filters. For brand-sample, N is applied separately to each selected brand.

```
effective_gold_sample_size = min(requested_gold_sample_size, eligible_silver_rows)
```

Safe convergence

An oversized Gold request does not fail and cannot invent rows. The task uses the eligible Silver population, emits a warning, and records both the requested and effective limits in the audit record.

RAW / BRONZE	SILVER	GOLD	DASHBOARD
review sample + related entities	fully featured selected reviews	chosen sample or aggregate	serving views from Gold

Bronze-to-Silver reference 1/3

Input, destination, execution, and workload controls for bronze_to_silver_yelp.

Purpose	Databricks Parameters JSON	Default / requirement
Help	"--help"	Optional
Google Drive folder	"--google-drive-folder-url", "{{job.parameters.google_drive_folder_url}}"	Use with connection
Google Drive connection	"--google-drive-connection", "{{job.parameters.google_drive_connection}}"	Required for Drive source
Volume input alternative	"--input-volume", "/Volumes/CATALOG/SCHEMA/VOLUME"	Do not combine source modes
Destination catalog	"--catalog", "workspace"	workspace
Destination schema	"--schema", "default"	default
Table prefix	"--table-prefix", "yelp"	yelp
Delta write mode	"--mode", "overwrite"	overwrite errorifexists
Spark partitions	"--partitions", "32"	Optional
Arrow batch ceiling	"--arrow-batch-size", "5000"	5000 records
Silver review limit	"--silver-sample-size", "25000"	All reviews if omitted
Silver seed	"--silver-sample-seed", "222"	222

Compatibility aliases

--sample-size and --sample-seed remain accepted. New Job definitions should use the explicit Silver names.

Bronze-to-Silver reference 2/3

Select full-corpus feature families by repeating `--nlp-component`. Omit it to run every standard group except transformers.

Feature group	Databricks Parameters JSON	What it adds
linguistic	"-nlp-component", "linguistic"	spaCy linguistic + structural features
vader	"-nlp-component", "vader"	VADER sentiment scores + labels
nrc-emotion	"-nlp-component", "nrc-emotion"	NRC emotion features
vad	"-nlp-component", "vad"	Valence, arousal, dominance
entities	"-nlp-component", "entities"	Named-entity features
grammar	"-nlp-component", "grammar"	Grammar + text-quality features
domain-lexicons	"-nlp-component", "domain-lexicons"	WorryWords, WCST, Yelp-domain features
time-weighting	"-nlp-component", "time-weighting"	Time-weighted analytical features
transformers	"-nlp-component", "transformers"	Optional local-model inference on compute

How pandas scales here

Spark partitions the review data and streams bounded Arrow batches into `mapInPandas`. pandas never receives the full dataset; peak pandas memory is governed by the Arrow batch ceiling on each executor.

Transformer boundary

Selecting transformers requires the model libraries and suitable compute on the task. It is not an API call and is deliberately excluded from the default feature set.

Bronze-to-Silver reference 3/3

Lexicon assets are small runtime dependencies. Keep them in an accessible Unity Catalog Volume, not in Git and not in the raw-data folder published with the repository.

Purpose	Databricks Parameters JSON	Requirement
Lexicon directory	"--lexicons-dir", "/Volumes/.../lexicons"	Resolve all assets from one folder
NRC VAD	"--vad-lexicon", "PATH/NRC-VAD-Lexicon-v2.1.txt"	Required for VAD
WorryWords	"--worry-lexicon", "PATH/worrywords-v1.txt"	Domain features
NRC WCST	"--wcst-lexicon", "PATH/NRC-WCST-Lexicon-v1.0.txt"	Domain features
Yelp AFFLEX/NEGLEX	"--yelp-lexicon", "PATH/yelp_lexicon.txt"	Yelp-domain features
NRC emotion intensity	"--nrc-intensity-lexicon", "PATH/NRC-Emotion-Intensity-Lexicon-v1.txt"	Emotion-intensity features

Literal terms stay literal

The loaders disable pandas' default NA-word conversion so vocabulary entries such as the English word 'null' remain real lexicon terms instead of becoming missing values.

Five exact source filenames

yelp_academic_dataset_review.json / business.json / user.json / checkin.json / tip.json - each using the full yelp_academic_dataset_ prefix.

Silver-to-Gold reference 1/3

Input, output, strategy, and deterministic row-sampling controls for silver_to_gold_yelp.

Purpose	Databricks Parameters JSON	Default / requirement
Help	"--help"	Optional
Catalog	"--catalog", "workspace"	workspace
Schema	"--schema", "default"	default
Table prefix	"--table-prefix", "yelp"	yelp
Explicit Silver input	"--input-table", "CATALOG.SCHEMA.yelp_silver_reviews"	Derived when omitted
Explicit Gold output	"--output-table", "CATALOG.SCHEMA.OUTPUT"	Derived when omitted
Gold strategy	"--gold-level", "brand-sample"	Required
Gold row limit	"--gold-sample-size", "5000"	Required for row-level sample
Gold seed	"--gold-sample-seed", "222"	222

Compatibility aliases

--sample-size and --sample-seed remain accepted. New Job definitions should use the explicit Gold names.

Silver-to-Gold reference 2/3

Choose one Gold grain, then repeat any cohort filters needed for that run.

GOLD LEVELS

Level	Output
brand-sample	One deterministic row-level review sample per selected brand
sample	One deterministic row-level review sample after all filters
business	Business-level analytical aggregates
user	User-level analytical aggregates
date	Time aggregates at the selected date granularity
industry	Industry-level analytical aggregates
sentiment	Sentiment-band analytical aggregates
emotion	Emotion-label analytical aggregates

REPEATABLE FILTERS

Purpose	Databricks Parameters JSON
Brands	"--brand", "Chipotle", "--brand", "Great Clips"
Business IDs	"--business-id", "ID_1", "--business-id", "ID_2"
Business names	"--business-name", "NAME_1", "--business-name", "NAME_2"
User IDs	"--user-id", "ID_1", "--user-id", "ID_2"
Industries	"--industry", "Restaurants", "--industry", "Beauty & Spas"
Cities / states	"--city", "Phoenix", "--state", "AZ"
Sentiment labels	"--sentiment-label", "positive"
Emotion labels	"--emotion-label", "joy"

Dashboard compatibility

brand-sample and sample preserve review rows and can feed the dashboard task. Business, user, date, industry, sentiment, and emotion are analytical aggregates and are not interchangeable with the dashboard source.

Silver-to-Gold reference 3/3

Date, star, sentiment, and emotion controls narrow the eligible Silver population before row sampling or aggregation.

Purpose	Databricks Parameters JSON	Default / choices
Inclusive start date	"--start-date", "2018-01-01"	Optional ISO date
Inclusive end date	"--end-date", "2021-12-31"	Optional ISO date
Date aggregation	"--date-granularity", "month"	day week month quarter year
Minimum stars	"--min-stars", "1"	Optional
Maximum stars	"--max-stars", "5"	Optional
Sentiment feature	"--sentiment-column", "vader_sentiment_score"	VADER, HF, Yelp lexicon, or Valence
Minimum sentiment	"--min-sentiment", "-0.25"	Optional
Maximum sentiment	"--max-sentiment", "1.0"	Optional
Emotion feature	"--emotion-column", "dominant_emotion"	dominant, lexicon, or HF label

Case study is configuration

The template selects Chipotle and Great Clips for the coursework case study. The task itself is reusable: replace the brands or select businesses, users, industries, geography, time windows, sentiment bands, or emotion labels.

Data-science dashboard layer

The third task does not choose another cohort. It translates the selected row-level Gold table into stable serving tables for presentation and analysis.

Purpose	Databricks Parameters JSON	Default / requirement
Catalog	"--catalog", "workspace"	workspace
Schema	"--schema", "default"	default
Table prefix	"--table-prefix", "yelp"	yelp
Selected Gold table	"--gold-table", "workspace.default.yelp_gold_dashboard_variant"	Use row-level Gold

REVIEW SAMPLE	BUSINESS SUMMARY	MONTHLY SUMMARY
Row-level detail Table: yelp_dashboard_review_sample Review text, stars, features, and Gold variant	Operational rollup Table: yelp_dashboard_business_summary Review counts, sentiment, risk, and attention tier	Trend layer Table: yelp_dashboard_monthly_summary Monthly review, rating, and sentiment movement

Direct dependency

Whatever Gold selects is what the dashboard represents. Change the brands, filters, dates, industries, or Gold sample size; then rerun the dependent dashboard task to refresh the serving layer.

Implementation architecture

The 2026 revision turns the coursework feature work into a reproducible distributed pipeline without publishing or reading the repository's local data directory.

Capability	Implemented design
Schemas	Explicit definitions for all five Yelp JSON datasets
Feature path	One canonical pandas implementation distributed through Spark mapInPandas
Writes	Spark-native Parquet, CSV, and Delta; no full driver-side collection
Validation	Mandatory at-scale validation before featured Silver publication
Audit	Failure-path logging plus queryable Delta audit records
Lexicons	Executor-safe resolution from local, Volume, or S3 sources
Parity	Schema, keys, nulls, strings, numerics, and row counts
Cloud shape	EMR packaging plus a three-task Databricks workflow
Presentation	Databricks dashboard-serving tables and Streamlit app layer

Production principle

Small pandas batches are an executor API, not a dataset destination. Spark owns partitioning, joins, persistence, validation, and distributed writes; pandas owns the canonical feature logic inside bounded Arrow batches.

Local release gate before cloud spend

The production pipeline is Databricks-only. This local Linux/WSL2 workflow is a verification gate for the shared feature code, dependencies, Spark writes, and package build.

RECOMMENDED: WSL2 / LINUX

```
git clone <this repo> ~/yelp_iter2
cd ~/yelp_iter2
./scripts/bootstrap_local.sh
```

Clone inside the Linux filesystem under ~/ - never below /mnt/c. The bootstrap requires Python 3.11 and Java 17 exactly, creates .venv, installs the pinned requirements and spaCy model, downloads the two required NLTK resources, validates dependencies, and runs the strict suite.

RERUN WITHOUT REBUILDING

```
PYSPARK_PYTHON="$PWD/.venv/bin/python" \
PYSPARK_DRIVER_PYTHON="$PWD/.venv/bin/python" \
./venv/bin/python ./scripts/run_test_suite.py
```

NATIVE WINDOWS: CONDITIONAL

```
powershell -ExecutionPolicy Bypass \
-File .\scripts\bootstrap_local.ps1
```

Use a short checkout path without parentheses, such as C:\dev\yelp_iter2. PySpark 3.5 launches through cmd.exe batch files; parentheses can surface as the misleading JAVA_GATEWAY_EXITED error.

Real local Parquet-write tests also require a trusted Hadoop build with %HADOOP_HOME%\bin\winutils.exe. The bootstrap does not download an unofficial executable. Use WSL2/Linux when no verified Windows build is already available.

Genuine pass

GENUINE PASS: N tests executed; 0 failures, 0 errors, 0 skips. Any skip, zero-test run, dependency drift, version mismatch, or missing NLP resource fails the gate.

Deterministic EMR package evidence

The 2026-09-20 gate built SHA-256 a6c83d49a024469d6814b9fe60a3b215abedcb004444bd3ec1be0059ddc5355d. The hash changes whenever a packaged pipeline module changes.

Verification record

The strict local gate verifies the shared feature path before cloud spend. It is evidence for release readiness, not a substitute for a Databricks canary.

16

TESTS EXECUTED

0

FAILURES

0

ERRORS

0

SKIPS

Strict gate - WSL2, 2026-09-20

Python 3.11 + Java 17 + pinned Spark/Arrow/NLP environment. A genuine pass requires tests to execute with zero failures, zero errors, and zero skips. Any skip, zero-test run, missing dependency, drifted version, or missing NLP resource is rejected.

End-to-end parity - 2026-09-03

Two disjoint 1,500-row coursework slices each matched 1,500 local rows to 1,500 distributed rows across all 68 expected feature columns, including schema, review keys, null positions, strings, numerics, and row counts.

Evidence boundary

The suite uses synthetic temporary fixtures and touches no corpus or cloud account. Passing it permits a small Databricks or EMR canary; it does not prove the full 8.6-million-review run. Cloud readiness becomes execution evidence only when the audit table records a successful live run.

Release path + rules to remember

The free verification work should precede the paid canary; the paid canary should precede scale.

BEFORE CLOUD SCALE

1

Run the strict release gate

Use the pinned Linux/WSL2 environment. Accept only the GENUINE PASS line with 0 failures, 0 errors, and 0 skips.

2

Start with a small Databricks canary

Use the same runtime, lexicons, scripts, source connection, and sample controls planned for the larger run.

3

Inspect the audit evidence

Check input and output counts, validation metrics, selected components, requested/effective sample sizes, and failures.

4

Inspect runtime behavior

Review executor memory, Arrow batch behavior, partition balance, and throughput before increasing workers.

5

Validate presentation permissions

Confirm the three dashboard-serving tables exist and the dashboard or app identity can read them.

6

Scale only after the canary

Increase data limits and workers deliberately; retain the same audit and validation gates.

KEEP THESE DISTINCTIONS

Rule	Meaning
DATABRICKS JSON	One array of quoted strings per Python-script task.
SILVER LIMIT	Controls reviews that receive joins and NLP.
GOLD LIMIT	Controls eligible row-level output after filters.
REPEATABLE FLAGS	Repeat the full flag/value pair every time.
RAW DATA	Google Drive, never Git.
LEXICONS	Unity Catalog Volume, never Git.
TRANSFORMERS	Optional compute dependency, not an API stage.
DASHBOARD	Always depends on the selected row-level Gold table.

Source of truth

This guide reflects the current three Databricks task interfaces, job_config.json, implementation architecture, and verified release-gate record. Rebuild it whenever executable parameters or validation evidence changes.

Verify the installation

From the repository root, the strict runner validates pinned dependencies and discovers tests/test_*.py:

```
./venv/bin/python ./scripts/run_test_suite.py | Accept only: GENUINE PASS ... 0 failures, 0 errors, 0 skips.
```